

Design Report: AXI-to-APB Bridge

1. Introduction & Purpose

The `Bridge_Complete` module is a **protocol converter** that sits between an **AXI-compliant master** (e.g., a CPU or DMA) and an **APB-compliant slave** (e.g., a low-power peripheral register file).

- **AXI side** (the master): Handles high-performance, pipelined memory requests.
- **APB side** (the slave): Handles simple, low-cost, non-pipelined register accesses.

Because the two protocols differ significantly in handshake and timing, the bridge acts as an **APB bus master** that accepts AXI-like transfer requests and executes them one by one over the APB bus.

2. The Protocols at a Glance

2.1 AXI (Advanced eXtensible Interface)

In this design, the AXI side is **simplified** and represented by **three FIFO interfaces**:

FIFO	Direction (Relative to Bridge)	Content
Request FIFO (<code>req_fifo</code>)	Input	A packet that bundles the target address and a write/read flag . The flag is the most significant bit (MSB) of the packet.
Write Data FIFO (<code>wr_fifo</code>)	Input	The actual write data and byte-enable strobes (PSTRB) that must be sent to the APB slave.
Read Data FIFO (<code>rd_fifo</code>)	Output	The read data returned from the APB slave, to be sent back to the AXI master.

The bridge consumes requests from these FIFOs using standard handshake signals (`_empty`, `_rd_en`, `_full`, `_wr_en`). This FIFO abstraction isolates the bridge from the full complexity of the AXI protocol (channels, bursts, ID tags, etc.).

2.2 APB (Advanced Peripheral Bus)

The APB side is a **true standard APB interface** (as defined by ARM®). The bridge drives the bus as a **master**. The signals are:

Signal	Direction (Relative to Bridge)	Function
PTRANSFER	Output	Enables the transfer. Must be 1 during the SETUP and ACCESS phases.
PWRITE	Output	1 for a write transfer, 0 for a read transfer.
PADDR	Output	The target address.
PWDATA	Output	Write data (valid when $PWRITE = 1$).
PSTRB	Output	Byte-lane strobes; indicate which bytes of PWDATA are valid.
PREADY	Input	Slave response. If 0, the slave inserts wait states and extends the ACCESS phase.
PRDATA	Input	Read data (valid when $PWRITE = 0$ and $PREADY = 1$).

APB transfers always consist of two cycles: **SETUP** (address valid) and **ACCESS** (data read/write), with optional wait states in the **ACCESS** phase.

3. Top-Level Architecture (Bridge_Complete)

The top module follows a classic **separation of control and data** structure. It contains two major sub-blocks:

1. **Bridge_CP (Control Path)** – Implements the finite-state machine (FSM), handshakes with the FIFOs, and issues the APB control strobes.
2. **Bridge_DP (Data Path)** – Handles all data routing, address mapping, and latching of read data.

This split makes the design easier to test and maintain. The **Bridge_Complete** simply wires the common clock (PCLK), reset (PRESET), FIFO signals, and APB signals to these two sub-blocks.

4. Control Path (Bridge_CP) – The Protocol Engine

The **Control Path** is the brain of the bridge. It contains a **six-state FSM** that orchestrates the exact sequence of a transaction.

4.1 FSM States

State	Description
S_IDLE	Wait for the AXI master to place a request. If req_fifo_empty is 0, assert req_fifo_rd_en and transition to S_REQ_DECODE.
S_REQ_DECODE	Peek at the MSB of the request packet. Latch it into a register (is_write_latched). If it is a write (1), go to S_WAIT_WDATA. If it is a read (0), go directly to S_SETUP.
S_WAIT_WDATA	Write only. Wait for the write data to arrive. When wr_fifo_empty becomes 0, assert wr_fifo_rd_en and proceed to S_SETUP.
S_SETUP	Drive the APB control signals (PTRANSFER = 1, PWRITE = is_write_latched). This matches the APB SETUP phase. Move immediately to S_ACCESS.
S_ACCESS	Keep PTRANSFER and PWRITE asserted. This is the APB ACCESS phase. Wait for the slave to assert PREADY. When it arrives: if it was a write , return to S_IDLE; if it was a read , go to S_STORE_RDATA.
S_STORE_RDATA	Read only. Ensure the Read Data FIFO is not full. If rd_fifo_full is 0, assert rd_fifo_wr_en and return to S_IDLE.

4.2 Handshaking with the FIFOs

- req_fifo_rd_en is pulsed **only once** in S_IDLE to pop the request.
- wr_fifo_rd_en is pulsed **only once** in S_WAIT_WDATA to pop the write data.
- rd_fifo_wr_en is pulsed **only once** in S_STORE_RDATA to push the read result.

This enforces that the bridge handles **one complete AXI request at a time**. The FSM does not allow a new request to be fetched until the previous APB transaction is fully finished.

5. Data Path (Bridge_DP) – The Wiring & Latching

The **Data Path** is purely combinational (with one latch) and routes the payload signals.

5.1 Request Packet Decoding

The request packet is $[ADDR_WIDTH:0]$ bits wide.

- `req_fifo_rd_data[ADDR_WIDTH-1:0]` → **PADDR** (the target address).
- `req_fifo_rd_data[ADDR_WIDTH]` → **is_write** (the type flag, also used inside the control path).

5.2 Write Data & Strobes

The write data packet is $[DATA_WIDTH + STRB_WIDTH - 1:0]$ bits wide.

- Lower `DATA_WIDTH` bits → **PWDATA**.
- Upper `STRB_WIDTH` bits → **PSTRB** (byte enables).

The data path passes these directly to the APB outputs.

5.3 Read Data Latching (Crucial APB Timing)

APB specifies that `PRDATA` must be valid when `PREADY` is high. However, the bridge needs to hold this read data for the `S_STORE_RDATA` state (which happens on the next clock edge after `PREADY` is seen).

To handle this, `Bridge_DP` contains a register `prdata_latched` that captures `PRDATA` **during the ACCESS phase** (when `state == S_ACCESS`). This latched value is then driven onto `rd_fifo_wr_data` in the subsequent state, ensuring the read data is stable before it is pushed into the FIFO.

6. Full Transaction Flow (Step-by-Step)

6.1 Write Transaction

1. **AXI Master** pushes a write request (address + flag 1) into `req_fifo`.
2. FSM: `S_IDLE` → (consumes request) → `S_REQ_DECODE`. Detects write.
3. FSM → `S_WAIT_WDATA`. Waits for master to fill `wr_fifo`.
4. **Master** pushes write data + strobes.
5. FSM sees `wr_fifo_empty = 0`, consumes the data, moves to `S_SETUP`.
6. **APB SETUP**: `PTRANSFER = 1`, `PWRITE = 1`. `PADDR`, `PWDATA`, `PSTRB` are valid.
7. FSM → `S_ACCESS`. Holds APB signals. Slave drives `PREADY`.

8. When `PREADY` is 1, FSM returns directly to `S_IDLE`. (Write completes on APB).
9. Bridge is ready for the next AXI request.

6.2 Read Transaction

1. **AXI Master** pushes a read request (address + flag 0) into `req_fifo`.
2. FSM: `S_IDLE` → (consumes request) → `S_REQ_DECODE`. Detects read.
3. FSM bypasses `S_WAIT_WDATA` and goes directly to `S_SETUP`.
4. **APB SETUP**: `PTRANSFER` = 1, `PWRITE` = 0. `PADDR` is valid.
5. FSM → `S_ACCESS`. Holds APB signals. Slave drives `PRDATA` and `PREADY`.
6. During `S_ACCESS`, the **Data Path** latches `PRDATA` into `prdata_latched`.
7. When `PREADY` is 1, FSM → `S_STORE_RDATA`.
8. **Data Path** drives `rd_fifo_wr_data` with the latched `PRDATA`. If `rd_fifo_full` is 0, FSM asserts `rd_fifo_wr_en` to push the data to the AXI master.
9. FSM returns to `S_IDLE`.

7. Design Strengths & Characteristics

- **Synchronous & Conservative**: The entire design is synchronous to `PCLK` with an active-low asynchronous reset (`PRESET`).
- **Single-threaded**: It handles one transaction at a time, making the logic simple and predictable – ideal for low-complexity bridges.
- **Parameterisable**: `ADDR_WIDTH` and `DATA_WIDTH` allow easy reuse across different bus widths (e.g., 32-bit, 64-bit).
- **Clean Separation**: The Control Path does not touch data signals; the Data Path does not decide the flow. This modularity aids functional verification.

8. Summary

In essence, `Bridge_Complete` acts as a **glue logic** component. It:

- **Decouples** the high-speed, burst-oriented AXI world from the low-speed, register-oriented APB world.
- **Serialises** incoming AXI-like requests into individual APB SETUP + ACCESS cycles.
- **Handles backpressure** via the FIFO signals (`_full` / `_empty`), preventing data loss when the APB slave or the AXI master is slow.

The RTL successfully implements the entire APB master protocol and provides a clean, FIFO-based interface for the AXI side, making it a robust building block for system-on-chip (SoC) interconnect fabrics.